Java Compile and Run Cheat Sheet

This focuses on **commands and workflows**, not Java syntax.

1. Compile one Java file

```
javac Hello.java
java Hello
```

This creates `Hello.class` in the same directory.

2. Compile class files into a separate directory

```
mkdir -p out
javac -d out Hello.java
java -cp out Hello
```

- `-d out` puts generated `.class` files inside `out/`.
- `-cp out` tells Java where to find the compiled classes.
- Do not write `.class` after the class name when running it.

3. Compile every Java file in the current directory

```
mkdir -p out
javac -d out *.java
java -cp out Main
```

Replace `Main` with the class containing:

```
public static void main(String[] args)
```

4. Compile Java files in a source directory

Example layout:

```
project/
├── src/
│   └── com/example/Main.java
└── out/
```

Compile all source files:

```
mkdir -p out
find src -name '*.java' -print0 | xargs -0 javac -d out
```

Run a class in a package:

```
java -cp out com.example.Main
```

Use the **fully qualified class name**: package name plus class name.

5. Compile with external libraries

Linux/macOS classpath separator: `:`

```
javac -cp 'lib/*' -d out $(find src -name '*.java')
java -cp 'out:lib/*' com.example.Main
```

Windows classpath separator: `;`

```
javac -cp "lib/*" -d out src\com\example\Main.java
java -cp "out;lib/*" com.example.Main
```

6. Detect whether compilation succeeded

```
rm -rf out
mkdir -p out

if javac -d out $(find src -name '*.java'); then
    echo "Compilation successful"
else
    echo "Compilation failed"
    exit 1
fi
```

`javac` returns:

- `0` — compilation succeeded
- non-zero — compilation failed

7. Detect whether a class can be run

Check that a compiled class file exists:

```
CLASS='com.example.Main'
FILE="out/${CLASS//./\/}.class"

if [ -f "$FILE" ]; then
    echo "$CLASS is compiled"
    java -cp out "$CLASS"
else
    echo "$CLASS was not compiled"
    exit 1
fi
```

8. Find compiled classes that contain a `main` method

After compilation, inspect every class with `javap`:

```
find out -name '*.class' ! -name '*$*' | while read -r file; do
    class=$(echo "$file" | sed 's#^out/##; s#/#.#g; s#\.class$##')

    if javap -classpath out "$class" 2>/dev/null | grep -q 'public static
        echo "Runnable class: $class"
    fi
done
```

Run the first detected runnable class:

```
main_class=$(find out -name '*.class' ! -name '*$*' | while read -r file;
    class=$(echo "$file" | sed 's#^out/##; s#/#.#g; s#\.class$##')

    if javap -classpath out "$class" 2>/dev/null | grep -q 'public static
        echo "$class"
        break
    fi
done)

if [ -n "$main_class" ]; then
    echo "Running $main_class"
    java -cp out "$main_class"
else
    echo "No class with main(String[]) found"
    exit 1
fi
```

9. Complete compile-and-run script

Save as `run-java.sh`:

```
#!/usr/bin/env bash
set -uo pipefail

SRC_DIR="${1:-src}"
OUT_DIR="${2:-out}"

rm -rf "$OUT_DIR"
mkdir -p "$OUT_DIR"

mapfile -d '' sources < <(find "$SRC_DIR" -name '*.java' -print0)

if [ "${#sources[@]}" -eq 0 ]; then
    echo "No Java source files found in $SRC_DIR"
    exit 1
fi

echo "Compiling ${#sources[@]} source file(s)..."

if ! javac -d "$OUT_DIR" "${sources[@]}"; then
    echo "Compilation failed"
    exit 1
fi

echo "Compilation successful"

main_class=""
while IFS= read -r -d '' file; do
    class=${file#"$OUT_DIR"/}
    class=${class//\//.}
    class=${class%.class}

    # Skip anonymous, inner, and synthetic classes.
    [[ "$class" == *\$* ]] && continue

    if javap -classpath "$OUT_DIR" "$class" 2>/dev/null | grep -q 'public
        main_class="$class"
        break
    fi
done < <(find "$OUT_DIR" -name '*.class' -print0)

if [ -z "$main_class" ]; then
    echo "Compilation succeeded, but no main(String[]) class was found"
    exit 1
```

```
fi

echo "Running $main_class"
java -cp "$OUT_DIR" "$main_class"
```

Make it executable and use it:

```
chmod +x run-java.sh
./run-java.sh src out
```

10. Compile and run with Java 11+ source-file mode

For a quick one-file test, without manually creating class files:

```
java Hello.java
```

This is convenient for small programs, but it does **not** replace the normal build workflow when you need all `.class` files kept in a chosen directory.

11. Clean generated class files

If classes were generated beside source files:

```
find . -name '*.class' -delete
```

If using a separate output directory:

```
rm -rf out
```

12. Useful diagnostic commands

Show the Java version:

```
java -version
javac -version
```

Show where Java is installed:

```
which java
which javac
```

Inspect a compiled class:

```
javap -classpath out com.example.Main
```

Inspect bytecode instructions:

```
javap -classpath out -c com.example.Main
```

Recommended standard workflow

```
rm -rf out
mkdir -p out
find src -name '*.java' -print0 | xargs -0 javac -d out && \
java -cp out com.example.Main
```

For a project with packages, keep the source layout and package names aligned:

```
src/com/example/Main.java
```

Compile to:

```
out/com/example/Main.class
```

Run with:

```
java -cp out com.example.Main
```